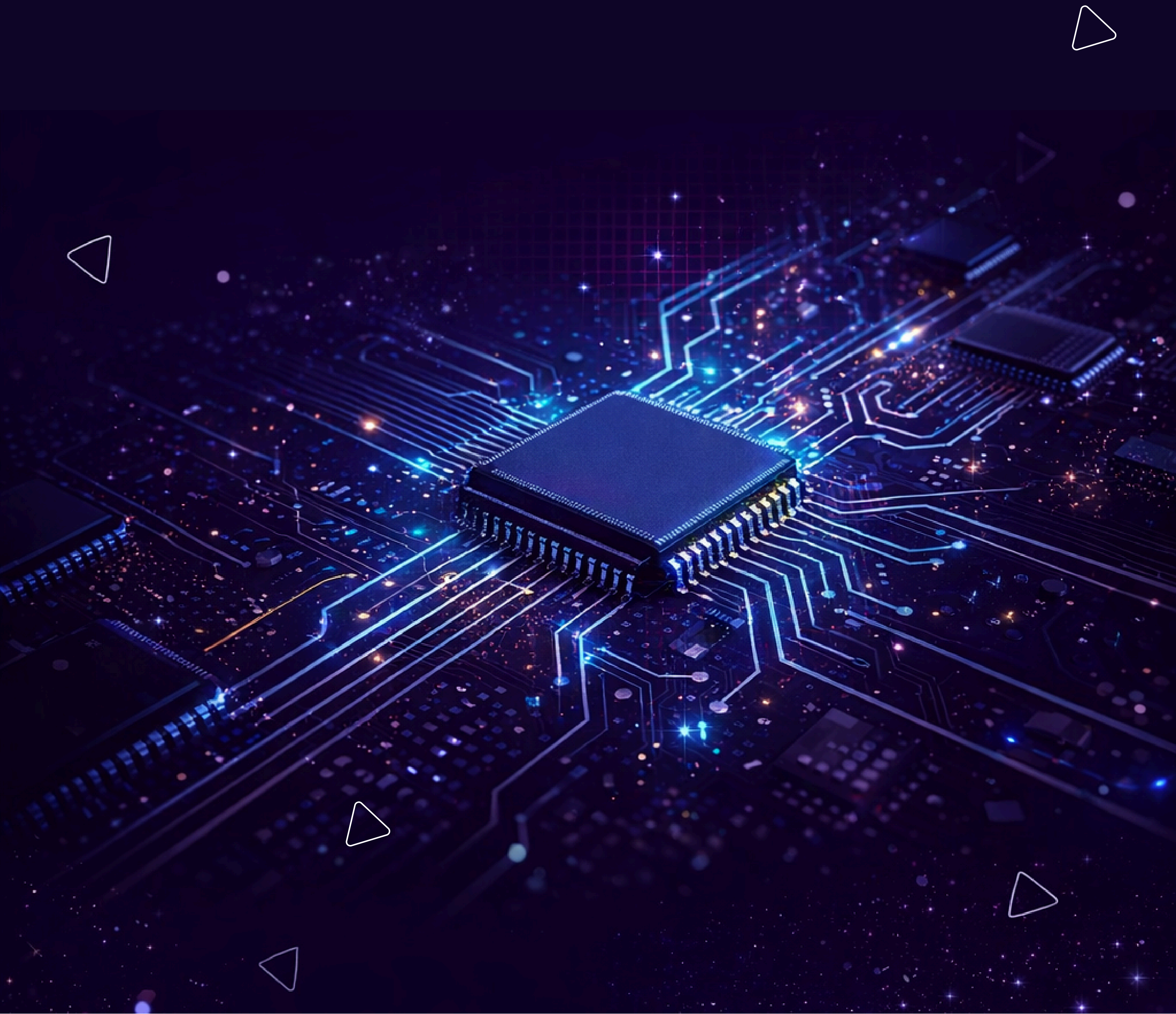


ADVANCED VERILOG INTERVIEW QUESTIONS

A CURATED COLLECTION OF REAL WORLD
PROBLEMS FOR DESIGN AND DEBUGGING
INTERVIEWS.



1. You are designing a 3 Stage pipeline in a DSP datapath.

```
always @(posedge clk) begin
    if (rst)
        stage1 <= 0;
    else
        stage1 <= in;
end

always @(posedge clk) begin
    stage2 = stage1;
end

always @(posedge clk) begin
    stage3 <= stage2;
end
```

During silicon testing, stage3 shows unstable values. What is the most likely reason?

- A) Reset is asynchronous
- B) Blocking assignment in stage2
- C) Missing sensitivity list
- D) Clock skew

2. A dual-clock FIFO is implemented as follows:

```
always @(posedge wr_clk)
    mem[wr_ptr] <= wr_data;

always @(posedge rd_clk)
    rd_data <= mem[rd_ptr];

always @(posedge wr_clk)
    rd_ptr_sync <= rd_ptr;
```

Occasionally, corrupted data is observed. What is the root cause?

- A) Memory depth insufficient
- B) Pointer synchronization incomplete
- C) Setup violation
- D) Read latency

3. A low-power design uses clock gating:

```
always @(posedge clk)
    enable_r <= enable;

assign gclk = clk & enable_r;

always @(posedge gclk)
    q <= d;
```

Random functional failures appear. What could be the main issue?

- A) Enable metastability
- B) Glitch on gated clock
- C) Hold violation
- D) Fanout

4. If in = 10 at cycle N, what is out at cycle N+2?

```
always @(posedge clk) begin
    if(rst)
        s1 <= 0;
    else
        s1 <= in;
end

always @(posedge clk)
    s2 = s1;
always @(posedge clk)
    out <= s2;
```

- A) 10
- B) X
- C) Previous value
- D) Unstable

5. If initially $a = 5$ and $b = 9$, what are the values after 1 clock cycle?

```
always @(posedge clk) begin
    a <= b;
    b <= a;
end
```

- A) $a = 9, b = 5$
- B) $a = 9, b = 9$
- C) $a = 5, b = 5$
- D) $a = 5, b = 9$

6. If initially $q = 7$ and $en = 0$ for 3 consecutive clock cycles, what will be the final value of q ?

```
always @(posedge clk) begin
    if (en)
        q <= d;
end
```

- A) 0
- B) X
- C) 7
- D) d

7. Consider the following datapath. If initially $s1=0$, $s2=0$, $s3=0$ and $in=1$ permanently, what is $s3$ after 3 clock cycles?

```
reg [7:0] s1, s2, s3;

always @(posedge clk) begin
    if (rst)
        s1 <= 8'd0;
    else
        s1 <= in + s3;
end

always @(posedge clk)
    s2 = s1 + 1;

always @(posedge clk)
    s3 <= s2;
```

- A) 1
- B) 2
- C) 3
- D) 4

8. If $data_in$ changes from $4'b0000$ to $4'b1111$ in $clkA$ domain, what is the most likely value observed at $sync2$?

```
reg [3:0] data_a;
reg [3:0] sync1, sync2;

always @(posedge clkA)
    data_a <= data_in;

always @(posedge clkB) begin
    sync1 <= data_a;
    sync2 <= sync1;
end
```

- A) Always 1111
- B) Always 0000
- C) May show intermediate values
- D) Compilation error

9. If we=1 and addr is same for read/write, what is returned in rdata?

```
reg [31:0] mem [0:15];  
reg [3:0] addr;  
  
always @(posedge clk) begin  
    if(we)  
        mem[addr] <= wdata;  
  
    rdata <= mem[addr];  
end
```

- A) New data
- B) Old data
- C) X
- D) Random

10. If set=1 and clear=1 simultaneously, what happens?

```
always @(posedge clk)  
    if(set)  
        ctrl <= 1;  
  
always @(posedge clk)  
    if(clear)  
        ctrl <= 0;
```

- A) ctrl = 1
- B) ctrl = 0
- C) Multiple driver conflict
- D) ctrl = X

11. What is the final value of a[i] dependent on?

```
integer i,j;  
always @(posedge clk) begin  
    for(i=0;i<4;i=i+1)  
        for(j=0;j<i;j=j+1)  
            a[i] <= a[j];  
end
```

- A) Old a[j] values only
- B) New updated values
- C) Mixed values
- D) Undefined

12. If en1 toggles faster than en2, what may occur?

```
always @(posedge clk)  
    if(en1)  
        q1 <= d;  
  
always @(posedge clk)  
    if(en2)  
        q2 <= q1;
```

- A) Data loss
- B) Overflow
- C) Latch inference
- D) Deadlock

13. What is the main issue in the given code?

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt <= 0;
    else if(en1)
        cnt <= cnt + 1;

always @(posedge clk)
    if(en2)
        cnt <= cnt - 1;
```

- A) Reset glitch
- B) Multiple drivers
- C) Priority inversion
- D) Metastability

14. What is latency from in to out in the following snippet?

```
reg d1,d2,d3;

always @(posedge clk) begin
    d1 <= in;
    d2 <= d1;
    d3 <= d2;
end

always @(posedge clk)
    out <= d3 & enable;
```

- A) 1 cycle
- B) 2 cycles
- C) 3 cycles
- D) 4 cycles

15. Find the hidden bug in the given logic.

```
parameter W=16;  
  
always @(*) begin  
    case(shift)  
        0: y = x;  
        1: y = {x[0],x[W-1:1]};  
        2: y = {x[1:0],x[W-1:2]};  
        3: y = {x[2:0],x[W-1:3]};  
    endcase  
end
```

- A) Latch inference
- B) Width mismatch
- C) Missing default case
- D) Wrong rotation logic

16. If both ports write to same address simultaneously, what is stored?

```
reg [15:0] mem [0:63];  
  
always @(posedge clkA)  
    if(weA)  
        mem[addrA] <= dataA;  
  
always @(posedge clkB)  
    if(weB)  
        mem[addrB] <= dataB;
```

- A) dataA
- B) dataB
- C) X / undefined
- D) Both

17. If in = 1 continuously and initially all registers are 0, what is acc after 5 clock cycles?

```
reg [7:0] acc, d1, d2;

always @(posedge clk) begin
    if(rst) begin
        acc <= 0;
        d1  <= 0;
        d2  <= 0;
    end
    else begin
        d1  <= in;
        d2  <= d1;
        acc <= acc + d2;
    end
end
```

- A) 3
- B) 4
- C) 5
- D) 6

18. If up=1 and down=1 together, what happens?

```
always @(posedge clk) begin
    if(up)
        cnt <= cnt + 1;
    if(down)
        cnt <= cnt - 1;
end
```

- A) cnt increments
- B) cnt decrements
- C) cnt unchanged
- D) cnt = X

19. What is the main problem in the given code snippet?

```
reg [7:0] r[0:7];

always @(posedge clk) begin
    if(rst) begin
        r[0]<=0; r[1]<=0;
    end
    else if(we)
        r[addr]<=data;
end
```

- A) Reset too slow
- B) Unreset registers
- C) Write hazard
- D) Multiple drivers

20. What is the main risk in the following snippet?

```
always @(*) begin
    crc = seed;
    for(i=0;i<8;i=i+1)
        if(data[i])
            crc = crc ^ poly;
end
```

- A) Latch inference
- B) Long combinational path
- C) Width mismatch
- D) Race

21. The following RAM returns old data during read-after-write. What is the correct explanation?

```
module ram (  
    input clk,  
    input we,  
    input [7:0] addr,  
    input [31:0] din,  
    output reg [31:0] dout  
);  
  
reg [31:0] mem [0:255];  
  
always @(posedge clk) begin  
    if (we)  
        mem[addr] <= din;  
  
    dout <= mem[addr];  
end
```

- A. Add pipeline register
- B. Implement write-through bypass
- C. Use blocking assignment
- D. Add asynchronous read

22. The following arbiter sometimes gives multiple grants. What is the bug?

```
module arbiter (  
    input [3:0] req,  
    output reg [3:0] grant  
);  
  
always @(*) begin  
    grant = 4'b0000;  
  
    if(req[0]) grant[0] = 1;  
    if(req[1]) grant[1] = 1;  
    if(req[2]) grant[2] = 1;  
    if(req[3]) grant[3] = 1;  
end  
  
endmodule
```

- A. Missing clock
- B. Priority not enforced
- C. Wrong datatype
- D. Missing reset

23. The following design shows metastability issues. What is the missing one?

```
module sync (  
    input clk_b,  
    input sig_a,  
    output reg sig_b  
);  
  
always @(posedge clk_b)  
    sig_b <= sig_a;  
  
endmodule
```

- A. Reset
- B. Enable signal
- C. Two-stage synchronizer
- D. FIFO

24. The following counter skips values. What is the cause?

```
always @(posedge clk) begin  
    if (rst)  
        cnt <= 0;  
    else begin  
        cnt <= cnt + 1;  
        cnt <= cnt + 2;  
    end  
end
```

- A. Reset error
- B. Multiple non-blocking assignments
- C. Wrong clock
- D. Width mismatch

25. What is the main issue in the given code snippet?

```
reg [1:0] state, next_state;

always @(posedge clk) begin
    if(rst)
        state <= IDLE;
    else
        state <= next_state;
end

always @(*) begin
    if(state == IDLE)
        next_state = RUN;
    else if(state == RUN)
        next_state = STOP;
end
```

- A. Missing default assignment
- B. Blocking assignment used
- C. Wrong reset polarity
- D. State width error

26. Why is output incorrect for the following code snippet?

```
reg [7:0] s1, s2, s3;

always @(posedge clk) begin
    if(rst) begin
        s1 <= 0;
        s2 <= 0;
        s3 <= 0;
    end else begin
        s1 <= in;
        s2 <= s1;
        s3 <= s2 + s1;
    end
end
```

- A. Race condition
- B. Wrong pipeline order
- C. Missing register stage
- D. Data dependency

27. What problem may occur for this buggy code snippet?

```
reg [7:0] mem [0:15];
reg [7:0] dout;

always @(posedge clk) begin
    if(we)
        mem[addr] <= din;

    if(re)
        dout <= mem[addr];
end
```

- A. Address overflow
- B. Read-after-write hazard
- C. Syntax error
- D. Width mismatch

28. What is unsafe here in this buggy code snippet?

```
wire gclk;  
  
assign gclk = clk & enable;  
always @(posedge gclk) begin  
    if(rst)  
        q <= 0;  
    else  
        q <= d;  
end
```

- A. Missing reset
- B. Combinational clock gating
- C. Wrong assignment
- D. Blocking usage

29. What is wrong in this buggy snippet of code?

```
reg [3:0] cnt;  
  
always @(posedge clk) begin  
    if(rst)  
        cnt <= 0;  
    else if(cnt == 10)  
        cnt <= 0;  
    else  
        cnt <= cnt + 1;  
end
```

- A. Width too small
- B. Wrong comparison
- C. Missing enable
- D. Reset error

30. Why is the output of the following code snippet incorrect?

```
always @(*) begin
    if(in[0])
        out = 0;
    if(in[1])
        out = 1;
    if(in[2])
        out = 2;
    if(in[3])
        out = 3;
end
```

- A. No default
- B. Missing case
- C. No priority
- D. Wrong width

31. Why does the pulse sometimes fail in the given code snippet?

```
reg d1;

always @(posedge clk) begin
    d1 <= sig;
    pulse <= sig & ~d1;
end
```

- A. Wrong operator
- B. Non-blocking delay
- C. Missing reset
- D. Width error

32. What is wrong in the below code snippet?

```
reg [3:0] wptr, rptr;  
  
assign full = (wptr == rptr);  
  
always @(posedge clk)  
    if(we)  
        wptr <= wptr + 1;  
  
always @(posedge clk)  
    if(re)  
        rptr <= rptr + 1;
```

- A. Wrong comparison
- B. Missing reset
- C. Pointer width
- D. No enable

33. Why 'eq' becomes X for the below code snippet?

```
reg [3:0] a, b;  
  
always @(*) begin  
    if(a == b)  
        eq = 1;  
end
```

- A. Missing else
- B. Wrong operator
- C. Width mismatch
- D. No clock

34. Why does out glitch in the given code snippet?

```
reg [1:0] state, next;
reg out;

always @(posedge clk) begin
    if(rst)
        state <= 2'b00;
    else
        state <= next;
end

always @(*) begin
    case(state)
        2'b00: next = 2'b01;
        2'b01: next = 2'b10;
        2'b10: next = 2'b00;
    endcase
end

always @(*) begin
    if(state == 2'b01)
        out = 1;
end
```

- A. Missing default assignment
- B. Wrong reset
- C. Blocking assignment
- D. Width mismatch

35. What is the main issue in the given code snippet?

```
reg [7:0] s1, s2, s3;
reg valid1, valid2, valid3;

always @(posedge clk) begin
    if(rst) begin
        s1 <= 0; s2 <= 0; s3 <= 0;
        valid1 <= 0; valid2 <= 0; valid3 <= 0;
    end
    else begin
        s1 <= in;
        valid1 <= in_valid;

        s2 <= s1;
        valid2 <= valid1;

        s3 <= s1 + s2;
        valid3 <= valid2;
    end
end
```

- A. Missing reset
- B. Wrong valid handling
- C. Data dependency
- D. Width error

36. What problem may occur for the given code snippet?

```
reg [7:0] mem [0:63];  
reg [7:0] dout_a, dout_b;  
  
always @(posedge clk) begin  
    if(we_a)  
        mem[addr_a] <= din_a;  
  
    if(we_b)  
        mem[addr_b] <= din_b;  
  
    dout_a <= mem[addr_a];  
    dout_b <= mem[addr_b];  
end
```

- A. Address overflow
- B. Write collision
- C. Latch inference
- D. Width mismatch

37. Why does the counter miscount for the given code snippet?

```
reg [7:0] cnt;  
reg en_d;  
  
always @(posedge clk) begin  
    en_d <= enable;  
end  
  
always @(posedge clk) begin  
    if(rst)  
        cnt <= 0;  
    else if(en_d)  
        cnt <= cnt + 1;  
    else  
        cnt <= cnt;  
end  
  
always @(*) begin  
    if(enable)  
        flag = 1;  
end
```

- A. Missing default
- B. Enable delay
- C. Width error
- D. Reset polarity

38. Why is the shifting incorrect for the given code snippet?

```
reg [15:0] shreg;  
  
always @(posedge clk) begin  
    if(rst)  
        shreg <= 0;  
    else if(load)  
        shreg <= din;  
    else begin  
        shreg[0]    <= sin;  
        shreg[15]   <= shreg[14];  
        shreg[14:1] <= shreg[13:0];  
    end  
end
```

- A. Bit overlap
- B. Wrong reset
- C. Width mismatch
- D. Missing enable

39. Why does done never assert for the given code snippet?

```
reg [2:0] state, next;

always @(posedge clk) begin
    if(rst)
        state <= 3'b000;
    else
        state <= next;
end

always @(*) begin
    case(state)
        3'b000: next = 3'b001;
        3'b001: next = 3'b010;
        3'b010: next = 3'b011;
        3'b011: next = 3'b100;
    endcase
end

assign done = (state == 3'b111);
```

- A. Missing terminal state
- B. Wrong reset
- C. Width mismatch
- D. Blocking assignment

40. What is wrong with the given code snippet?

```
reg [7:0] s1, s2, s3;  
reg v1, v2, v3;  
  
always @(posedge clk) begin  
    if(rst) begin  
        s1<=0; s2<=0; s3<=0;  
        v1<=0; v2<=0; v3<=0;  
    end else begin  
        s1 <= in;  
        v1 <= in_valid;  
  
        s2 <= s1;  
        v2 <= v1;  
  
        s3 <= s2 * 2;  
        v3 <= v1;  
    end  
end
```

- A. Wrong valid propagation
- B. Missing reset
- C. Data overflow
- D. Width mismatch

41. What is main issue with the given code snippet?

```
reg [31:0] rf[0:15];  
reg [31:0] rd;  
  
always @(posedge clk) begin  
    if(we)  
        rf[wa] <= wd;  
  
    if(re)  
        rd <= rf[ra];  
end  
  
always @(posedge clk) begin  
    if(clear)  
        rf[ra] <= 0;  
end
```

- A. Read conflict
- B. Multiple write drivers
- C. Width mismatch
- D. Reset missing

42. Why data shifts wrongly in the given code snippet?

```
reg [7:0] shreg;  
reg [2:0] bitc;  
  
always @(posedge sclk) begin  
    if(start) begin  
        shreg <= din;  
        bitc <= 0;  
    end  
    else begin  
        shreg <= shreg << 1;  
        bitc <= bitc + 1;  
    end  
end  
  
assign mosi = shreg[7];
```

- A. Missing reset
- B. Wrong shift edge
- C. Counter overflow
- D. Width mismatch

43. What is wrong in the code snippet for Asynchronous Input ?

```
reg sync1, sync2, sync3;

always @(posedge clk) begin
    sync1 <= async_in;
    sync2 <= sync1;
end

always @(posedge clk) begin
    sync3 <= async_in;
end

assign out = sync3 & sync2;
```

- A. Missing reset
- B. Improper synchronizer
- C. Width error
- D. Timing violation

44. Find the bug in the given code snippet.

```
reg [1:0] state, next;

always @(posedge clk) begin
    if(rst)
        state <= 2'b00;
    else
        state <= next;
end

always @(*) begin
    case(state)
        2'b00: next = 2'b01;
        2'b01: next = 2'b10;
        2'b10: next = 2'b11;
    endcase
end
```

- A. Missing default case
- B. Wrong reset value
- C. Width mismatch
- D. Blocking assignment

45. Why is r3 incorrect for the given code snippet?

```
reg [7:0] r1, r2, r3;

always @(posedge clk) begin
    if(rst) begin
        r1 <= 0;
        r2 <= 0;
        r3 <= 0;
    end
    else begin
        r1 <= in;
        r2 <= r1;
        r3 <= r1 + r2;
    end
end
```

- A. Missing stage
- B. Dependency issue
- C. Width error
- D. Reset polarity



**Excellence in World class
VLSI Training & Placements**

Do follow for updates & enquires



+91- 9182280927